QuizTimer4Zoom
System Architecture Diagram

Secure Zoom Application Architecture
OAuth 2.0 PKCE | Serverless Backend | Canvas Rendering

Architecture Layers

Layer 1: Zoom Meeting Context

- User authenticated in Zoom meeting
- Permission to use QuizTimer4Zoom app
- Video feed available for rendering

Layer 2: Frontend (Browser)

- quiztimer.html - Main HTML container
- Canvas 2D API - Timer rendering with dynamic font sizing
- Zoom SDK v0.16.19 - Video overlay and context API
- Settings UI - Position, size, color controls with localStorage

Layer 3: Backend (Node.js / Express.js)

- OAuth 2.0 PKCE - Secure authentication with code verifier
- Session Manager - Encrypted secure cookies (httpOnly, secure, sameSite)
- Helmet.js - Security headers (CSP, HSTS, X-Frame-Options)
- Input Validation - All endpoints validated, no sensitive data logging

Layer 4: External APIs

- Zoom OAuth Endpoints - /authorize, /token
- Zoom Meeting API - User info and context
- Vercel Platform - Serverless hosting with auto-scaling

Layer 5: Data Storage

- Encrypted Session Cookies - User authentication, 24h expiration
- Browser localStorage - Settings persistence
- Zoom Context Header - AES-256-GCM encrypted validation

OAuth 2.0 PKCE Authentication Flow

1. Installation
   User clicks "Install" !' Redirected to /install endpoint
   Server generates random 128-character code_verifier
   Server creates code_challenge = base64url(SHA256(code_verifier))
   Verifier stored in secure httpOnly session cookie
2. Authorization Request
   User redirected to Zoom OAuth with code_challenge parameter
   Zoom shows user consent screen requesting app permissions
   User authenticates with Zoom credentials
3. User Consent
   User grants requested scopes and permissions
   Zoom generates short-lived authorization code
   Zoom redirects back to /auth callback with authorization code
4. Token Exchange (PKCE Protection)
   Server receives authorization code from callback
   Server validates state parameter (CSRF protection)
   Server retrieves stored code_verifier from session
   Server sends to Zoom: code, code_verifier, client_id, client_secret
   Zoom validates code_verifier matches code_challenge
   Zoom cannot be spoofed - requires matching verifier
5. Session Creation
   Zoom returns access_token (JWT, ~1 hour)
   Server creates encrypted secure session:
   - httpOnly: true (JavaScript cannot access)
   - secure: true (HTTPS only)
   - sameSite: 'Strict' (CSRF protection)
   - maxAge: 24 * 60 * 60 * 1000 (24-hour expiration)
6. App Access
   Server redirects to Zoom deeplink to open app in meeting
   Browser launches app in Zoom with authenticated context
   App receives encrypted x-zoom-app-context header
   Server validates and decrypts context (AES-256-GCM)
   App is now running with secure session

Timer Operation & Data Flow

TIMER START

1. User clicks "Start" button in QuizTimer UI
2. JavaScript records startTime = Date.now()
3. setInterval begins 1-second loop

EACH SECOND

1. Calculate: elapsedSeconds = (Date.now() - startTime) / 1000
2. Calculate: secondsRemaining = duration - elapsedSeconds
3. Determine color state:
   - if secondsRemaining > 5: normal color
   - if secondsRemaining "d 5: warning color (orange)
   - if secondsRemaining "d 0: timeout color (gray)

RENDER TO CANVAS

1. Clear canvas: ctx.clearRect(0, 0, width, height)
2. Calculate optimal font size using binary search
   - Find largest font that fits canvas dimensions
   - Performance: ~7-8 iterations, O(log n)
3. Set font and color: ctx.font = size + 'px Arial'
4. Draw number: ctx.fillText(secondsRemaining, x, y)
5. Get ImageData: ctx.getImageData(0, 0, width, height)

SEND TO ZOOM

1. Call zoomSdk.drawImage({
   imageData: ImageData,
   x: position.x,
   y: position.y,
   width: settings.width,
   height: settings.height
   })
2. Zoom overlays ImageData on meeting video feed
3. Timer visible to all meeting participants
4. Previous image cleared automatically

TIMER COMPLETE

1. When secondsRemaining "d 0: stop interval
2. Timer stays at 0 (optional: play notification sound)
3. User must manually reset or start new timer

Security Implementation

AUTHENTICATION SECURITY

- OAuth 2.0 with PKCE prevents authorization code interception
- Code verifier proves client legitimacy without exposing client secret
- Secure session cookies: httpOnly (XSS), secure (HTTPS), sameSite=Strict (CSRF)
- No password storage - delegated to Zoom

NETWORK SECURITY

- HTTPS/TLS 1.2+ enforced on all connections
- Perfect forward secrecy enabled
- HSTS header (max-age=31536000) forces HTTPS
- X-Content-Type-Options: nosniff prevents MIME sniffing
- X-Frame-Options: DENY prevents clickjacking
- Content-Security-Policy limits resource loading

DATA PROTECTION

- Minimal data collection: user ID, email, auth token only
- No logging of sensitive data or tokens
- Session data auto-deleted after 24 hours
- AES-256-GCM encryption for Zoom context validation
- Input validation on all API endpoints

DEPENDENCY SECURITY

- All direct dependencies scanned with npm audit
- 0 critical vulnerabilities in application code
- 100% of direct dependencies verified secure
- Monthly security review cycle
- Critical patches applied within 24 hours

COMPLIANCE

- GDPR compliant: user rights, data minimization, consent
- CCPA compliant: user rights, no data sales
- OWASP Top 10: all major categories addressed
- NIST Framework: aligned with all functions
- Zoom Marketplace: exceeds all security requirements